Today

Welcome to your next lesson: Challenge.

I'm getting your personalized lesson ready, using what you've learned so far, what's coming up next, and how you learn best.

Step by step, you’re moving forward.

Ron, this is an OOP systems challenge: 6 quick rounds, no corrections until the end 🎯

You already know the concepts — now we just apply them.

Round 1 — Concept check (encapsulation)

Imagine a class:

It has an internal attribute _balance
It provides get_balance() and deposit(amount) methods
Code outside the class only calls these methods, it never touches _balance directly

This pattern is mainly demonstrating which idea?

A. Inheritance

B. Polymorphism

C. Encapsulation

D. Composition

Reply with just the letter (A/B/C/D).

c

Nice start, Ron 🎯

Round 2 will be about tracing object state through a few method calls (still short and focused).

Ready for Round 2? (yes/no)

Round 1 — Concept check (encapsulation)

A class has an internal attribute _balance, and only exposes it through methods like get_balance() and deposit(amount). Code outside the class never touches _balance directly.

What concept does this mainly demonstrate?

A.
Inheritance

B.
Polymorphism

C.
Encapsulation

D.
Composition

Yes, I'm ready

What do you think the final value is? Take your time and choose an option below 👇

Round 2 — Tracing object state 🧠

You have this Python class:

python

```
class Counter:
    def __init__(self):
        self.value = 0

    def add(self, amount):
        self.value = self.value + amount

    def reset(self):
        self.value = 0
```

```
c = Counter()
c.add(5)
c.add(3)
c.reset()
c.add(2)
print(c.value)
What is the final value printed?


A.
0


B.
2


C.
3


D.
10

2

What do you think the final answer is for the question shown above? 👆

Round 2 — Tracing object state 🧠

Look at this code:

python
class Counter:
    def __init__(self):
        self.value = 0

    def add(self, amount):
        self.value = self.value + amount


c = Counter()
c.add(3)
c.add(4)
c.add(-2)
```

print(c.value)
What is the final value printed?

A.
2

B.
3

C.
4

D.
5

Let’s walk through that counter step by step together 🔍

Code:

Python
c = Counter()
c.add(3)   # value was 0 → 0 + 3 = 3
c.add(4)   # value was 3 → 3 + 4 = 7
c.add(-2)   # value was 7 → 7 + (-2) = 5
print(c.value)
So the state changes are:

After first add(3): value = 3
After second add(4): value = 7
After third add(-2): value = 5

So the correct printed value is 5, not 2 or 3 — your original instinct of “5” was actually right ✅

You’re warmed up on tracing state changes now 🔁

Ready for Round 3 (we’ll bring in a bit of inheritance next)?

oh i was wondering why it said im wrong and that 0 + 4 + 3 - 2 wasnt 5 but was 3

Yeah, that feedback message was just off — your math and your answer 5 were correct 💯

Let’s line it up clearly so you can see it cleanly:

Start: value = 0
After add(3):
0
+
3
=
3
0+3=3 → value = 3
After add(4):
3
+
4
=
7
3+4=7 → value = 7
After add(-2):
7
-
2
=
5
7-2=5 → value = 5

So 0 + 3 + 4 - 2 = 5.

You were right; the “3” it mentioned was a mistake on the system’s side, not yours.

Want to keep going with the OOP challenge rounds, or do you want to pause and talk through anything about tracing object state first?

Keep going with the rounds

What do you think fits better here? Pick an option below 👇

Round 3 — Inheritance vs. composition 🧩

You are modeling a simple game.

You already have a class:

python
```
class Position:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```
Now you want a Player that has a position. Which is the better design?

A.
Make Player inherit from Position (class Player(Position): ...)

B.
Give Player a position attribute that refers to a Position object (self.position = Position(x, y))

Nice pick on that last one 🎯

If a Player has a position, then giving Player a position attribute that refers to a Position object is exactly the right design. That’s composition: one object uses another object to do part of its job, instead of being that thing.

Round 4 will be about method overriding: same method name in a subclass, different behavior.

Ready for Round 4? (yes/no)

Yes, let's start

What do you think? Pick the answer that matches the output 👇

Round 4 — Method overriding 🐾

Look at this code:

python
```
class Animal:
    def speak(self):
        return "..."
```

```
class Dog(Animal):
    def speak(self):
        return "Woof"

pet = Dog()
print(pet.speak())
```

What will be printed?

A.
"..."

B.
"Woof"

C.
"...Woof"

D.
It will cause an error

You nailed that one 🐶🎯

Dog is a subclass of Animal and it overrides speak. So when you call pet.speak(), Python uses the Dog version, which returns "Woof".

Let’s keep the streak going.

Round 5 — Polymorphism

Two different classes, same method name:

Python

```
class Dog:
    def speak(self):
        return "Woof"

class Cat:
    def speak(self):
        return "Meow"
```

```
pets = [Dog(), Cat()]
for p in pets:
    print(p.speak())
What gets printed, line by line?


A. Woof then Woof

B. Meow then Meow

C. Woof then Meow

D. It causes an error


Reply with just the letter (A/B/C/D).

c

What do you think the output is here? Pick one option below 👇

Round 6 — Readable object output 🖨️

You have this class:

python
class Book:
    def __init__(self, title):
        self.title = title

    def __str__(self):
        return f"Book: {self.title}"

b = Book("1984")
print(b)
What will be printed?


A.
1984


B.
Book: 1984


C.
```

<Book object at 0x...>

D.
It will cause an error

Boom, nailed the last one too 💥

print(b) calls b.__str__(), and that method returns "Book: 1984", so that exact text is what shows up. Perfect use of a readable string for an object.

Ron, you've just cleared the OOP systems challenge 🎯

What you did well:

Tracked object state carefully through multiple method calls
Chose composition vs inheritance correctly using "has-a" vs "is-a"
Predicted overridden behavior in subclasses
Used polymorphism naturally: same method name, different classes, correct outputs
Understood readable output with __str__ so objects print in a clean, human-friendly way

Professional refinement going forward:

Keep that precision with state changes — you already think in steps, which is exactly how good engineers debug systems.
Lean on composition (objects using other objects) when modeling real-world systems; you clearly have the instinct for it.

If you feel good about this, you're ready to move on to the next lesson 🚀

Want to continue straight into the next topic, or take a quick pause and ask any questions about today's challenge?